SASC: Soft-Averaged Self-Consistency to Improve Chain-of-Thought Reasoning in Instruct-LLMs

Ruban Vishnu Pandian [1] **Alliot Nagle** [1] **Hyeji Kim** [1]

Abstract

Improving the reasoning performance of LLMs during inference by combining scores of multiple output reasoning traces has emerged as a prominent technique, particularly for Chain-of-Thought Instruct-LLM models. Methods such as Self-Consistency and Self-Certainty assign simple yet effective metrics to the generated outputs which are then used to determine the final answer. These methods fit into the framework of designing a score vector over the set of generated answers, using weighted averaging of one-hot hard-decision vectors. In this work, we propose *Soft-Averaged Self-Consistency*, a simple method in which the hard-decision vectors are replaced with the conditional probability vectors obtained from the LLM logits, forming our soft-decision vectors. We show that our theoretically motivated method provides improved accuracy when compared against other baselines, backed by extensive experiments on a variety of tasks and models.

1. Introduction

Chain-of-Thought prompting (Wei et al., 2023) involves instructing an LLM to generate reasoning steps before arriving at a final answer. Building on this framework, significant research has been conducted to improve LLM performance during inference by adding extra test-time computation, particularly for instruction-tuned LLMs. Best-of-N selection methods (Snell et al., 2024) involve sampling multiple reasoning traces for a given input, assigning scores to each trace post-inference, and choosing the final answer from the score-maximizing trace. Methods such as Self-Consistency (Wang et al., 2023) and Self-Certainty (Kang et al., 2025) build on this framework and decide the final answer by assigning scores to the generated output answers. In Self-Consistency, the answer is selected via simple majority voting. Self-Certainty uses rank-based Borda voting.

In these methods, scores are aggregated with the answers treated as hard decisions. In other words, the score assigned to an answer is simply the sum of the individual scores assigned to those reasoning traces that produced that answer. Therefore, the probability information of getting other possible answers given a reasoning trace is not used. Two traces that produced the same answer are equally weighed, even though the probability of generating that answer could be different across those traces.

We found that Self-Consistency is equivalent to finding the empirical estimate of the marginal answer probability and then selecting the answer that maximizes it, i.e., the mode of the distribution. Based on this insight, we propose **Soft-Averaged Self-Consistency (SASC)**, a novel idea in which the scores are used in conjunction with the probability vectors that can be easily obtained from the LLM logits for each reasoning trace. In short, the scores are soft-averaged to improve performance, as shown in Figure 1.

To validate our idea across various tasks such as mathematical reasoning, code reasoning and general reasoning, we extensively benchmark it on the following datasets: GSM8K (Cobbe et al., 2021), MATH-500 (Lightman et al., 2023), CRUXEval (Gu et al., 2024) and GPQA-Main (Rein et al., 2023). Our results demonstrate that soft-averaging the scores, which leverages the inherent uncertainty the models have in their own answers, leads to better accuracy across nearly all benchmarks and hyperparameter settings.

The key properties of SASC are listed below:

- **Theoretical grounding:** We rigorously derive how using soft-decision vectors provides a better estimate of the answer distribution rather than the hard-decision vectors. Assuming the model has low bias, selecting the final answer from this estimate will yield better performance.
- **Robustness:** SASC improves accuracy across different datasets, models, hyperparameter values, and reasoning budgets by exploiting the model's uncertainty in its own answers.

[1]Chandra Department of Electrical and Computer Engineering, University of Texas at Austin, USA. Correspondence to: Ruban Vishnu Pandian <rubanvp@utexas.edu>.

Accepted to the Structured Probabilistic Inference & Generative Modeling workshop at ICML 2026, Seoul, South Korea. PMLR 306, 2026.

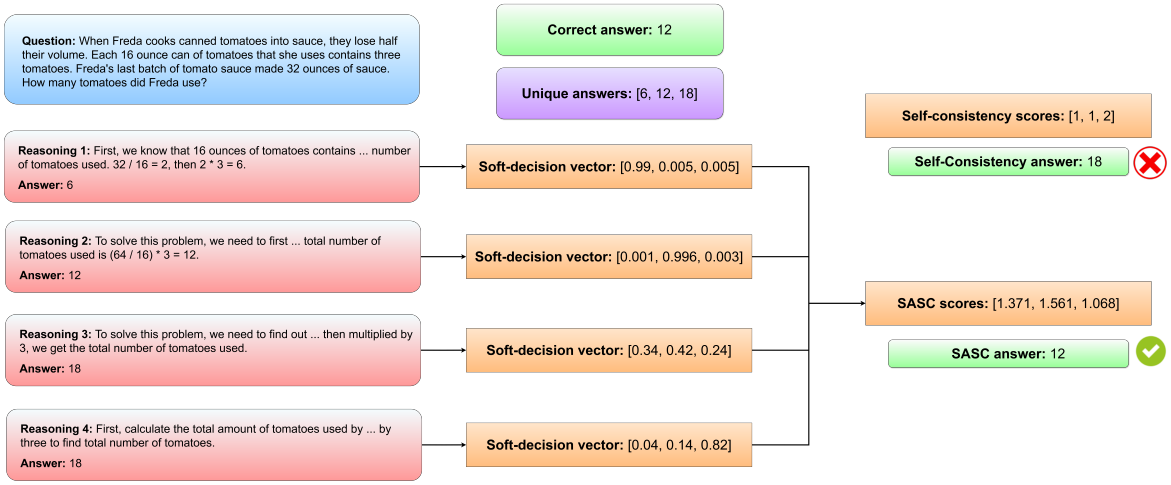

Figure 1. SASC uses soft-decision vectors to choose the final answer. Here is a real example from the MATH-500 dataset, showing how SASC identifies the correct answer, compared to vanilla Self-Consistency. The soft-decision vectors contain the conditional probabilities of getting the unique answers given the respective reasoning sequences. For example, even though the output answer is 18 in the third trace, its probability is actually 0.24. SASC uses this information to obtain a more accurate estimate of the answer distribution.

- **Applicability:** SASC is built on existing score-based weighted averaging frameworks such as Self-Consistency and Self-Certainty and can therefore be easily integrated with other scoring frameworks with a similar underlying structure.

2. Related Work

Best-of-N Selection using Reward Models: In Best-of-N selection (Snell et al., 2024), multiple reasoning traces are sampled using an LLM, a score is assigned to each trace using a reward model, and the trace maximizing this score is chosen as the final output trace. Process reward models (PRM) (Lightman et al., 2023) assign scores based on the whole reasoning trace, unlike outcome reward models (ORM), which assign scores only based on the final answer. However, these reward models are typically both task-specific and computation-heavy, limiting their scalability.

Scalable, Light-Weight Methods for Best-of-N Sampling: Low-complexity reward functions, utilizing the LLM's internal logits to encode its own certainty of its answers, have been proposed in the literature. The authors of Self-Consistency (Wang et al., 2023) show that sampling multiple reasoning traces and then choosing the answer via majority voting significantly improves performance for Chain-of-Thought models. Self-Certainty (Kang et al., 2025) builds on this idea by utilizing the internal logits to assign a confidence metric known as Self-Certainty, to a reasoning trace. The generated traces are then ranked accordingly, and the answer is selected using the Borda voting rule. Both methods require minimal computational overhead since no external reward models are used.

Reasoning-pruning Perplexity Consistency (Zhou et al., 2025) directly aggregates the probabilities of the reasoning traces to form a score function over the set of answers. Low-probability answers are filtered out, and the final answer is chosen as the one that maximizes this probability score. In all these methods, the answers are treated as hard decisions. For a generated output trace, the final answer associated with that trace is treated as the only possible answer given that trace.

Majority of the Bests: Benefits of Best-of-N selection and Self-Consistency are combined in the Majority of the Bests method (Rakhsha et al., 2025). It is an ensemble averaging method in which the set of output reasoning traces is sampled with replacement to obtain multiple bootstrap datasets. Best-of-N selection is then applied on each of these datasets to obtain a list of intermediate answers over which majority voting is done to obtain the final answer. This method primarily reduces the variance of Best-of-N selection.

Soft Metrics for Self-Consistency: As opposed to majority voting-based vanilla Self-Consistency, prior work has been done in developing soft metrics to determine the final answer (Wang et al., 2024). Three internal probability-based metrics have been proposed: Mean, Min, and Product of token probabilities of a candidate reasoning trace. Although the titles of our work and this paper sound similar, we would like to emphasize that our framework is fundamentally different. In this work, they choose the final answer using Best-of-N selection based on one of the three metrics mentioned above: they select the reasoning trace that maximizes that metric and extract the final answer from it. However, in our approach, we **soft-average** these score metrics using the

probability information, a fundamentally different operation than a direct Best-of-N approach. Further mathematical details of these different approaches are explained in the upcoming sections.

3. Background

3.1. LLM Background

LLMs are auto-regressive models that operate on a sequence of input tokens and produce a sequence of output tokens. Formally, let the input sequence be $x = (x_1,\ x_2,\ x_3,\ ...,\ x_n)$. Then the LLM outputs an output sequence $y = (y_1,\ y_2,\ y_3,\ ...,\ y_m)$ where each output token is generated sequentially from a distribution produced by the LLM. It is denoted as $p(\cdot|x, y_{<i})$ where $i \in \{1, 2, ..., m\}$ and $y_{<i} = (y_1,\ y_2,\ y_3,\ ...,\ y_{i-1})$ ($y_{<i}$ is empty for the first output token, i.e., for $i = 1$). Formally, $p(\cdot|x, y_{<i})$ is a distribution endowed upon V where V is the vocabulary these tokens belong to.

3.2. Best-of-N Methods

Let $z = f(y)$ denote the answer extraction function for an output token sequence y. Typically, in mathematical/code reasoning datasets, it is usually a subsequence of tokens that appears at the end of the reasoning trace. For all practical purposes, we can assume the answer extraction function just extracts this subsequence without any significant additional processing. The reasoning trace can then be thought of as $y = (r, z)$ where r is the reasoning subsequence and z is the final answer following it. This setup is especially true for Instruct-LLMs, which can be prompted to produce an output with reasoning steps followed by a final answer. Best-of-N selection methods sample multiple such traces, i.e., samples $(r_1, z_1),\ (r_2, z_2),\ ...\ ,\ (r_N, z_N)$, for the same input x. Then a score $g(r, z)$ is computed for each of these samples. Finally, the answer is declared as:

$$z_{\text{final}} = z_k \text{ where } k = \text{argmax}_{i \in 1,2,...,N}\ g(r_i, z_i)$$

3.3. Self-Consistency and Self-Certainty

These methods do not directly fit into the Best-of-N framework. In these methods, a unique ensemble of answers A is formed, i.e., from $z_1,\ z_2, ...,\ z_N$, each distinct answer appears only once in A. Let $A = \{a_1,\ a_2,\ ...,\ a_M\}$ (clearly $M \leq N$). A score function $s_{\text{method}} : A \to \mathbb{R}$ is defined for a given method. For Self-Consistency, it is simply the count of occurrences of these answers, formally given as:

$$s_{\text{self-cons}}(a) = \sum_{n=1}^{N} \mathbb{I}(z_n = a);\ \ a \in A$$

where $I(\cdot)$ denotes the indicator function. In Self-Certainty, the score is computed in a two-step process. First, each reasoning trace is assigned a confidence metric as defined below (Kang et al., 2025):

$$\textbf{Self-Certainty} = -\frac{1}{m|V|} \sum_{i=1}^{m} \sum_{j=1}^{|V|} \log(|V| \cdot p(v_j|x, y_{<i}))$$

where v_j is a placeholder dummy variable denoting the elements of the vocabulary and y denotes the output token sequence. It is simply the average of token-wise KL divergences between the next token distributions and the uniform distribution over the vocabulary. Once these metrics are computed for each reasoning trace, they are ranked and arranged in decreasing order of this metric. For a reasoning trace at the r^{th} position in this order, i.e., rank r, the score is given as $(N - r + 1)^p$. Formally, the score for the n^{th} sampled reasoning trace is given as:

$$W(n) = (N - r(n) + 1)^p$$

where $r(n)$ denotes its rank and p is a hyperparameter. Now that the per-trace score is defined, the score function for the unique answers is defined as:

$$s_{\text{self-cert}}(a) = \sum_{n=1}^{N} W(n)\mathbb{I}(z_n = a);\ \ a \in A$$

Note that setting $p = 0$ reduces the above method to Self-Consistency.

4. Our Contribution

4.1. Mathematical Motivation

We begin by noting that both the above score functions can be written in a general form given below:

$$s(a) = \sum_{n=1}^{N} W(n)\mathbb{I}(z_n = a);\ \ a \in A$$

where $W(n)$ is the per-trace weight of the n^{th} trace for a particular method. We call this the **Weighted Averaging of One-hot Vectors (WOHV)** framework. In general, any prior work that follows an answer-matching-based score aggregation strategy falls into this framework. For these methods, the score vector can be formed as follows:

$$s = [s(a_1),\ s(a_2),\ ...,\ s(a_M)]^T$$

Note that s is exactly the same as:

$$s = \sum_{n=1}^{N} W(n)\mathbb{I}_{z_n}$$

where $\mathbb{I}_{z_n} \in \{0, 1\}^M$ denotes the one-hot unit vector with the 1 appearing at position z_n and 0s everywhere else. It is

also easy to see that $\mathbb{I}_{z_n}$ is a valid probability distribution over the unique ensemble of generated answers. These vectors exactly show why output answers are treated as hard decisions in these methods, since they are assigned a probability of occurrence of 1.

However, we have direct access to the probability of these answers. Specifically, for each reasoning trace r_n, we have access to:

$$p_{r_n} = [p(a_1|r_n),\ p(a_2|r_n),\ ...,\ p(a_M|r_n)]^T$$

which we call **Soft-decision vectors** since they assign probabilities to the answers rather than treating them as hard decisions. We assume the dependency of the answers on the input prompt x is implicit and hence, omit x in the above expression for notational convenience.

Let us consider the Self-Consistency method, i.e., $W(n) = 1$. In this case, the score vector is $s = \sum_{n=1}^{N} \mathbb{I}_{z_n}$. We define a scaled version of this score vector and also another vector from the soft-decision vectors as follows:

$$p_1 = \frac{1}{N}\sum_{n=1}^{N} \mathbb{I}_{z_n}; \quad p_2 = \frac{1}{N}\sum_{n=1}^{N} p_{r_n}$$

It turns out both p_1 and p_2 are tractable, unbiased estimates of the true marginal PMF vector of the answer candidates, endowed upon by the LLM, which is mathematically given as:

$$p_{\text{true}}(a) = \sum_{r \in \text{SR}} p(a|r)p(r); \quad a \in A$$

$$p_{\text{true}} = [p_{\text{true}}(a_1),\ p_{\text{true}}(a_2),\ ...,\ p_{\text{true}}(a_M)]^T$$

where SR is the set of all possible reasoning traces. Based on this perspective of distribution estimation, let us define two loss functions related to these estimates:

$$L(p_1) = D_{\text{KL}}(p_1||p_{\text{true}}); \quad L(p_2) = D_{\text{KL}}(p_2||p_{\text{true}})$$

where $D_{\text{KL}}(\cdot||\cdot)$ denotes the standard KL divergence. As one might expect, p_2, which exploits additional information, yields a smaller KL divergence loss than p_1. In Appendix A, we derive an explicit bound on this gap, the main result of which is presented below.

Key Proposition: Using the notation introduced above, we have the following result:

$$\mathbb{E}[L(p_1)] - \mathbb{E}[L(p_2)] \geq \frac{1}{2N}\left(1 - \mathbb{E}\left[T(r)\right]\right) \qquad (1)$$

where $T(r) = \sum_{z \in \mathcal{Z}} p(z|r)^2$ is the squared ℓ_2 norm of the conditional distribution vector of the answers given the reasoning trace r and $\mathcal{Z}$ denotes the set of all possible answers. The right-hand side, known as Jensen's gap, quantifies the advantage of soft-averaging (used in p_2) over hard-averaging (used in p_1). If most soft-decision vectors are highly dispersed (i.e., close to the uniform distribution), then their corresponding $T(r)$ values would be low, leading to a higher Jensen's gap. In contrast, when the soft-decision vectors are one-hot (i.e., exhibit no randomness), then $T(r) = 1$ for those, resulting in a lower gap.

4.2. Our Method: Soft-Averaged Self-Consistency

Key proposition 1 serves as the **main theoretical motivation** for our method since it implies that when $W(n) = 1$, i.e., in the Self-Consistency method, determining the final answer from the sum of soft-decision vectors is better than doing it from the sum of hard-decision vectors. For a general weighing function $W(n)$, we propose to use the following modified score function:

$$\hat{s} = \sum_{n=1}^{N} W(n) p_{r_n}$$

The final answer is then simply chosen as the one which maximizes this score, i.e., the position of the maximum score in this vector. More details regarding the applicability of soft-averaging for a general weighing function $W(\cdot)$ are provided in Appendix A.5. We empirically show that for the Self-Consistency and Self-Certainty baselines, using the soft-decision vectors provides better performance. Implementation details regarding soft-vectors computation are presented in algorithm 1, latency overheads of computing soft-vectors are presented in Appendix B.1, and analysis of bias-variance components of the error of the SASC method is presented in Appendix A.4.

5. Experimental Setup

We evaluate our methods across four datasets: GSM8K (Cobbe et al., 2021), MATH-500 (Lightman et al., 2023), CRUXEval (Gu et al., 2024) and GPQA-Main (Rein et al., 2023) spanning math, code and general reasoning. Our baselines are the aforementioned Self-Consistency (Wang et al., 2023), Self-Certainty with voting (Kang et al., 2025) and Majority of the Bests (Rakhsha et al., 2025) methods. We compare the performance of our soft-decision-based methods with their hard-decision-based baselines. The methods were also tested across different values of the number of outputs $N = 8, 16, 32, 64$ and Borda vote ranking power values $p = 0.4, 1.2, 2.0$.

The default LLM hyperparameter values used for inference are temperature $= 0.7$ and top-p $= 1$. The ZeroEval framework (Lin, 2024) for evaluating the zero-shot performance of Instruct-LLMs was used as the base framework on the HuggingFace engine. With slight modifications, the source codes from the Self-Certainty paper (Kang et al., 2025) were

integrated with this framework for conducting all the experiments in this paper. All the experiments were carried out on NVIDIA GeForce RTX 5090 GPUs.

6. Results

In this section, we empirically demonstrate that SASC improves reasoning performance, indicating that the theoretical bounds derived for distribution estimation translate into practical gains in reasoning accuracy.

6.1. Unnormalized vs Normalized Soft Averaging

The elements of the soft-decision vector p_{r_n} for a given reasoning trace r_n should ideally sum to 1 over all possible answers. However, in practice, not all answers are generated by the samples, and hence, the vector p_{r_n} is said to be unnormalized, i.e., its elements need not sum to 1. This is in contrast to hard-decision one-hot vectors, which, by definition, always sum to 1. Hence, a third score variant was also tested in the experiments, which is given below:

$$\hat{s}_{norm} = \sum_{n=1}^{N} W(n)\tilde{p}_{r_n}; \quad \tilde{p}_{r_n} = \frac{1}{\sum_{b\in A} p_{r_n}(b)} p_{r_n}$$

Doing this normalization ensures that the soft-decision vectors describe a valid distribution over the set of generated unique answers A while also being scaled versions of the original probability vectors p_{r_n} obtained from the LLM's internal logits. We propose this as a computationally light variant; it can be tested alongside the unnormalized version, and the variant that performs better can be used.

We would also like to emphasize the differences between our methods and the unnormalized/normalized weighted-sum methods mentioned in the Self-Consistency paper (Wang et al., 2023). In this paper, the authors study the performance of the Self-Consistency method for $W(n) = p(r_n, z_n)$, which is referred to as the unnormalized weighted sum method. Similarly, they define the normalized weighted-sum method in which $W(n)$ is the probability normalized in the logarithmic domain. However, this still falls into the WOHV framework, i.e., score vector is of the form: $s = \sum_{n=1}^{N} W(n)\mathbb{I}_{z_n}$. Our key contribution is using probability information about the likelihood of obtaining the answer given a reasoning trace across the entire set of generated answers, rather than just the answer corresponding to that particular reasoning trace.

6.2. Performance Variation across Number of Outputs N and Borda Power p

SASC was tested against the mentioned baselines across various benchmarks under different hyperparameter settings. In particular, the variations across number of outputs (N) and Borda voting parameter (p) using the Llama-3.2-3B-Instruct model (Grattafiori et al., 2024) are presented below.

Table 1. Accuracies compared across Self-Consistency and Self-Certainty methods for Llama-3.2-3B-Instruct. Soft-averaging consistently improves the accuracy by 1-2%.

Method	**GSM8K**		**MATH-500**		**CRUXEval**		**GPQA-Main**	
	$N=8$	$N=64$	$N=8$	$N=64$	$N=8$	$N=64$	$N=8$	$N=64$
Self-Consistency	77.33	82.79	44.60	51.00	34.25	37.13	28.57	29.69
SASC **(Ours)**								
• Unnormalized	**79.18**	**83.75**	46.39	52.31	34.29	**39.25**	29.53	30.58
• Normalized	76.86	82.75	**47.48**	**52.73**	**36.63**	**39.25**	**29.81**	**30.96**
Self-Certainty								
- $p=0.4$	77.48	83.09	45.80	51.80	35.63	37.63	31.25	29.46
- $p=1.2$	76.12	83.17	46.00	52.80	35.50	37.75	**31.70**	29.02
- $p=2.0$	74.91	83.40	45.40	53.20	34.38	38.25	29.69	29.69
SASC **(Ours)**								
• Unnormalized								
- $p=0.4$	**79.03**	83.98	46.88	53.71	35.67	39.50	29.74	31.30
- $p=1.2$	78.21	84.51	46.69	53.51	35.54	39.38	31.04	30.12
- $p=2.0$	77.22	**84.66**	46.99	53.51	34.79	**40.25**	31.12	31.27
• Normalized								
- $p=0.4$	77.81	82.98	48.09	53.97	**36.13**	39.75	29.40	**32.01**
- $p=1.2$	76.92	83.36	**48.18**	**54.58**	36.00	39.63	30.83	31.08
- $p=2.0$	75.47	83.66	48.07	53.56	35.00	39.88	30.98	31.41

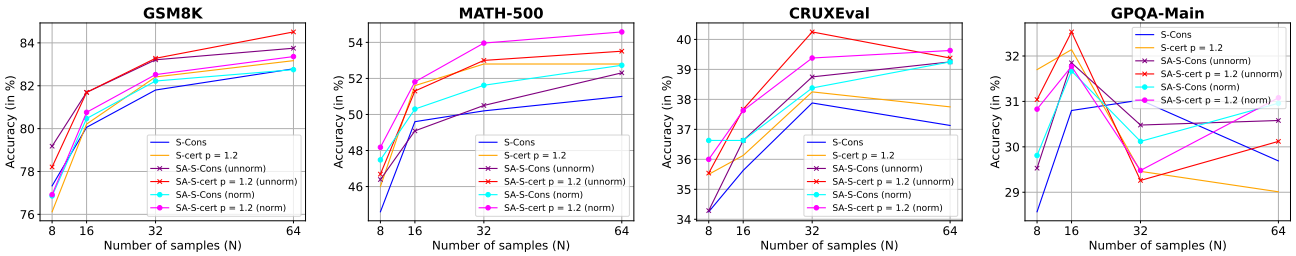

Figure 2. Performance of different methods for $N = 8, 16, 32, 64$ using Llama-3.2-3B-Instruct. One of the soft-averaging-based methods provides the highest accuracy in nearly all cases. SA methods in the figures correspond to our soft-averaging-based variants.

Table 1 shows how soft-averaging improves performance for both the Self-Consistency and the Self-Certainty baselines for different datasets across various settings of N and p values. Figure 2 further demonstrates the scalability of soft-averaging over different N values. Further results using the Llama-3.1-8B-Instruct model (Grattafiori et al., 2024) are present in Appendix C.1.

6.3. Performance Improvement over the Majority of the Bests Baseline

SASC is a general method that can be applied to any answer-scoring method that follows the WOHV framework. To demonstrate the same, we conducted experiments using the Majority of the Bests **(MoB)** (Rakhsha et al., 2025) method. It is a recent, ensemble method, based on the concept of bootstrap aggregation. In this method, multiple Bootstrap datasets are obtained by uniformly and independently sampling the set of output reasoning traces with replacement. Best-of-N selection is done based on a reward on each bootstrap dataset to obtain a list of best reasoning traces, and finally, Self-Consistency is applied on this list to obtain the final answer.

In Appendix A.6, we derive how the score function assigned to the set of answers by MoB fits the WOHV framework. This is true because even in MoB, the answers present at the end of reasoning traces are considered as hard final answers instead of probabilistic outputs. Hence, we show that using SASC, the performance of the MoB method can be improved. To demonstrate this, we tested both MoB and soft-averaged MoB on Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct models for $N = 8, 64$, using the Self-Certainty metric as the reward to perform Best-of-N selection. We used $m = \lfloor \sqrt{N} \rfloor$ as the number of samples to be chosen per bootstrap dataset, an idea directly taken from (Rakhsha et al., 2025).

Table 2 shows that in nearly all cases, SASC provides noticeable performance improvement over the MoB baseline, highlighting its versatility. It is not dependent on the base method; as long as the base method fits the WOHV framework, SASC will provide an improvement by using the rich information present in the soft-decision vectors.

Table 2. Accuracies compared across MoB and soft-averaged MoB for Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct models. Soft-averaging provides noticeable improvement over the MoB baseline.

Method	**GSM8K**		**MATH-500**		**CRUXEval**		**GPQA-Main**	
	$N = 8$	$N = 64$	$N = 8$	$N = 64$	$N = 8$	$N = 64$	$N = 8$	$N = 64$
Llama-3.2-3B-Instruct								
MoB	75.02	82.47	45.64	52.66	33.75	38.13	31.05	29.10
SA-MoB **(Ours)**								
• Unnormalized	**77.15**	**83.89**	46.99	52.60	34.54	39.42	30.88	28.96
• Normalized	75.40	82.43	**47.67**	**53.14**	**34.75**	**39.88**	**31.71**	**29.32**
Llama-3.1-8B-Instruct								
MoB	90.60	91.28	54.80	**62.00**	45.50	47.88	**32.96**	**35.71**
SA-MoB **(Ours)**								
• Unnormalized	90.14	91.21	54.80	61.20	46.50	48.88	32.59	34.82
• Normalized	**90.75**	**91.36**	**55.60**	61.80	**46.63**	**49.00**	32.21	34.82

6.4. Performance Variation across Reasoning Token Budgets

Using standard prompting templates from the ZeroEval framework (Lin, 2024) with minimal modifications, the models were prompted to produce a reasoning and a final answer, with constraints on the number of tokens they could use for reasoning before producing the final answer tokens. The reasoning budgets were enforced softly through prompts using the template provided in (Han et al., 2025). The following line was added to the prompt to soft-enforce a reasoning budget:

> Use less than **max_think_tokens** tokens for reasoning. The final "answer" token must be generated immediately after that.

where **max_think_tokens** is a variable denoting the reasoning tokens budget. In addition to the standard **No limit** case (in which, the extra line is not added to the prompt), we tested our methods with token budgets of 100 and 500 to examine their performance at low token budgets. We hypothesized that when the reasoning budget is constrained, the models might not be fully certain about their outputs, in which case our methods can perform better by utilizing that uncertainty. Figure 3 demonstrates the advantage SASC provides under different reasoning budgets.

All the results presented so far were based on the Llama3 family of models. We also studied the performance of SASC using Qwen models. We experimented with and without reasoning budgets using Qwen models and observed that, without any reasoning budget constraints, they were highly certain in their answers, meaning their soft-decision vectors were nearly one-hot. In such cases, SASC will be similar to the Self-Consistency baseline and deliver only marginal improvement or performance similar to the baselines. Below we have presented the results of SASC on Qwen models under budgeted scenarios. We tested the Qwen-2.5-3B-Instruct and Qwen-2.5-7B-Instruct models (Team, 2024) with hyperparameters $N = 16$ and reasoning budgets $R = 100, 500$. The Borda vote parameter p, which provided the highest accuracy in each case, was chosen as the representative value.

Table 3 demonstrates that SASC mostly provides a performance improvement, and if not, it at least has similar

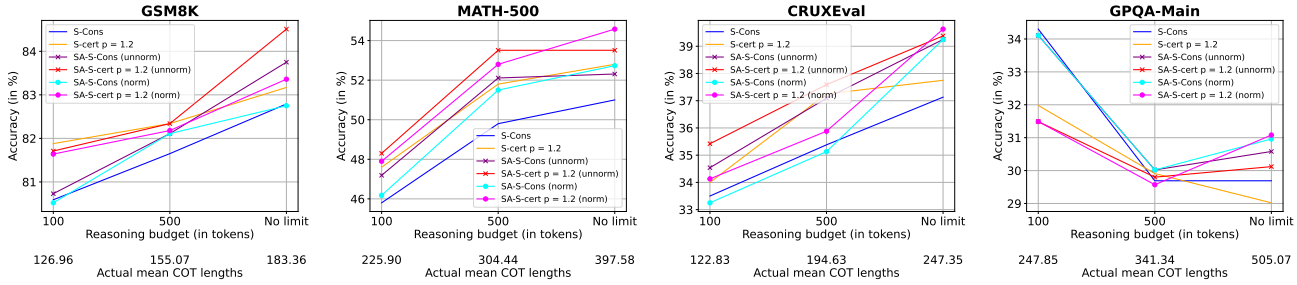

Figure 3. Performance of different methods for reasoning budgets $100, 500$ and "No limit" case using Llama-3.2-3B-Instruct. One of the soft-averaging-based methods provides the highest accuracy in nearly all cases. SA methods in the figures correspond to our soft-averaging-based variants.

Table 3. Accuracies compared across Self-Certainty and SASC for Qwen-2.5-3B-Instruct and Qwen-2.5-7B-Instruct models under reduced reasoning budgets

Method	**GSM8K**		**MATH-500**		**CRUXEval**		**GPQA-Main**	
	$R = 100$	$R = 500$	$R = 100$	$R = 500$	$R = 100$	$R = 500$	$R = 100$	$R = 500$
Qwen-2.5-3B-Instruct								
Self-Certainty	74.68	84.53	34.80	47.20	38.00	42.88	**26.74**	28.57
SASC (**Ours**)								
• Unnormalized	**75.96**	**85.22**	**35.60**	**48.30**	**38.63**	**43.75**	26.24	**29.24**
• Normalized	75.13	84.45	35.00	46.69	37.50	43.00	26.24	29.01
Qwen-2.5-7B-Instruct								
Self-Certainty	**87.18**	92.95	54.60	68.20	**53.13**	**58.38**	36.62	**35.49**
SASC (**Ours**)								
• Unnormalized	**87.18**	92.95	**55.20**	**68.40**	52.75	58.00	36.62	35.27
• Normalized	87.11	**93.03**	54.40	68.20	**53.13**	58.13	**36.92**	35.27

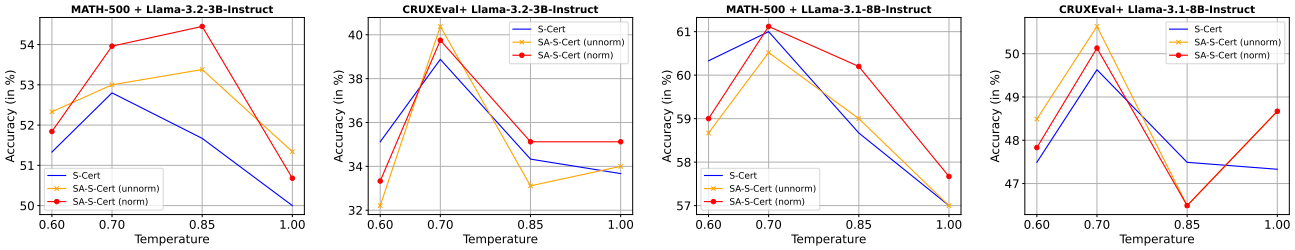

Figure 4. Performance of different methods for $T = 0.60, 0.70, 0.85, 1$ using Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct models. One of the soft-averaging-based methods provides the highest accuracy in nearly all cases. SA methods in the figures correspond to our soft-averaging-based variants.

performance as the baselines, highlighting its robustness. We also motivate why these scenarios are important from a computational point of view in Appendix B.2. Further analysis of performance variations across models is presented in Appendix C.2.

6.5. Ablations across Sampling Temperatures

The soft-decision vectors are formed from the conditional probabilities of answers given the reasoning traces, which depend on the inference sampling temperature. To ensure the robustness of SASC across different sampling temperatures, further experiments were conducted. In particular, we benchmarked the MATH-500 and CRUXEval datasets using the Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct models with $N = 32$. The Borda vote parameter p providing the highest accuracy in each case was chosen as the representative value. Figure 4 demonstrates that one of the SASC methods provides improvement in performance over the Self-Certainty baseline in nearly all cases, demonstrating the robustness of SASC.

7. Limitations and Future Research

Soft-averaging is a lightweight method with minimal computational overhead. Our findings suggest that it consistently improves over baselines across various tasks and hyperparameter values, positioning it as a scalable method that can be easily integrated into any scoring method following the WOHV framework.

However, similar to the Self-Consistency baseline, soft-averaging estimates the answer distribution endowed upon the set of answer by the LLM and selects the final answer from it, meaning its performance is limited by the model's inherent bias. Future work can explore applying soft-averaging during model training or fine-tuning to mitigate this bias. In particular, SASC-based score functions can be incorporated as effective reward functions in RL-based fine-tuning methods such as GRPO. Moreover, as with the baselines we have considered in this work, soft-averaging too relies on answer-matching-based score aggregation. Future work can expand soft-averaging to open-ended generation tasks where score aggregation cannot be based solely on answer matching.

8. Conclusions

In this paper, we introduce Soft-Averaged Self-Consistency (SASC), a novel, lightweight method that we have demonstrated improves the performance of Chain-of-Thought Instruct-LLMs by providing a better estimate of the final answer distribution. We relate the Self-Consistency framework to a distribution estimation problem and theoretically show that, by using soft-decision vectors, the distribution can be estimated more accurately. We use that result to motivate replacing the one-hot hard-decision vectors with the soft-decision conditional PMF vectors in the weighted averaging of one-hot vectors based scoring framework. We show empirical improvement in performance over different datasets, models, hyperparameter settings, and reasoning budgets for the Self-Consistency, Self-Certainty, and Majority of the Bests baselines, demonstrating the robustness of our method. With minimal computational overhead, it can be effectively integrated into similar scoring frameworks and improve performance by exploiting the model's inherent uncertainty in its own answers.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgements

This work was partially supported by NSF CAREER Award 2443857, ARO Award W911NF2310062, ONR Award N000142412542, and the 6G@UT center within the Wireless Networking and Communications Group (WNCG) at the University of Texas at Austin.

References

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta,

S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Gu, A., Rozière, B., Leather, H., Solar-Lezama, A., Synnaeve, G., and Wang, S. I. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024. URL https://arxiv.org/abs/2401.03065.

Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., and Chen, Z. Token-budget-aware llm reasoning, 2025. URL https://arxiv.org/abs/2412.18547.

Hayashi, M. Exponential decreasing rate of leaked information in universal random privacy amplification. *IEEE Transactions on Information Theory*, 57(6):3989–4001, June 2011. ISSN 1557-9654. doi: 10.1109/tit.2011.2110950. URL http://dx.doi.org/10.1109/TIT.2011.2110950.

Kang, Z., Zhao, X., and Song, D. Scalable best-of-n selection for large language models via self-certainty, 2025. URL https://arxiv.org/abs/2502.18581.

Liao, J. G. and Berg, A. Sharpening jensen's inequality, 2017. URL https://arxiv.org/abs/1707.08644.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL https://arxiv.org/abs/2305.20050.

Lin, B. Y. ZeroEval: A Unified Framework for Evaluating Language Models, July 2024. URL https://github.com/WildEval/ZeroEval.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

Rakhsha, A., Madan, K., Zhang, T., massoud Farahmand, A., and Khasahmadi, A. Majority of the bests: Improving best-of-n via bootstrapping, 2025. URL https://arxiv.org/abs/2511.18630.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL https://arxiv.org/abs/2408.03314.

Team, Q. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Team, Q. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Wang, H., Prasad, A., Stengel-Eskin, E., and Bansal, M. Soft self-consistency improves language model agents, 2024. URL https://arxiv.org/abs/2402.13212.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models, 2023. URL https://arxiv.org/abs/2203.11171.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

Zhou, Z., Tan, Y., Li, Z., Yao, Y., Guo, L.-Z., Li, Y.-F., and Ma, X. A theoretical study on bridging internal probability and self-consistency for llm reasoning, 2025. URL https://arxiv.org/abs/2510.15444.

A. Appendix - Mathematical Analysis

In this section, we will understand why using soft decision vectors are better than hard decision vectors from a distribution estimation perspective.

A.1. Self-Consistency as a Distribution Estimation and Mode Retrieval Method

Let us first mathematically formulate the problem statement at hand. We are given a joint PMF (Probability mass function) $p(X = x, Y = y)$ operating on the space $\mathcal{X} \times \mathcal{Y}$ where $\mathcal{X}, \mathcal{Y}$ denote the supports of random variables X, Y respectively. This distribution is sampled independently N times to form the dataset:

$$S = \{(x_1, y_1), (x_2, y_2), ..., (x_N, y_N)\}$$

Let us also define $S_x = \{x_1,\ x_2,\ ...,\ x_N\}$ and $S_y = \{y_1,\ y_2,\ ...,\ y_N\}$ for notational convenience. The aim is to estimate the mode of the marginal PMF $p(Y = y)$ i.e., find the value:

$$y_{\text{mode}} = \text{argmax}_{y \in \mathcal{Y}}\ p(Y = y)$$

Let p_{true} denote the true marginal distribution vector of the random variable Y defined over $\mathcal{Y}$. Then the above problem statement is the same as finding the position of the highest value in this vector.

In our problem, the RVs X and Y denote the reasoning trace r and final answer z respectively (defined in Section 3.2). p_{true} is then simply the vector containing the marginal PMF values $p_{\text{LLM}}(z)$ endowed by the LLM on the final answer RV z. Practically, it is impossible to find this marginal PMF since LLMs produce the output tokens auto-regressively, i.e., without producing the reasoning sequence r first, it is not possible to get any probability information about the final answer z. And the number of possible output reasoning sequences is enormous (A reasoning trace of size 50 with only the top 10 tokens considered for sampling at each step already spans a sample set of size 10^{50}).

The Self-Consistency method approaches this problem in a simple, straight-forward manner by using the samples to form an empirical distribution and then reporting its mode as the answer. Formally, the empirical distribution vector can be written as:

$$p_1 = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}y_n$$

where $\mathbb{I}y_n$ is the one-hot vector as defined in 4.1. And the final answer is chosen as the $y \in \mathcal{Y}$ corresponding to the highest element in this vector.

A.2. Comparison of the Hard and Soft Decision Vectors

However we have more information at hand that can be leveraged to form a better estimate of the distribution. Note that from the LLM logits, the conditional PMF $p(Y = y|X = x)$ can be computed. Based on this assumption, let us look into the next estimate of the distribution which is given as:

$$p_2 = \frac{1}{N} \sum_{n=1}^{N} p(\cdot|x_n)$$

where $p(\cdot|x_n)$ denotes the conditional PMF vector over $\mathcal{Y}$ conditioned on x_n. With p_1 and p_2 defined, some of the properties they follow are proved below:

Theorem A.1 (Both p_1 and p_2 are unbiased estimators of the marginal PMF vector p_{true})**.**

Proof. Writing p_1 and p_2 in index notation, we have:

$$p_1(y) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(y_n = y); \quad p_2(y) = \frac{1}{N} \sum_{n=1}^{N} p(Y = y|x_n)$$

for $y \in \mathcal{Y}$. Here $\mathbb{I}(y_n = y)$ denotes the Bernoulli indicator RV which is 1 if the condition inside is satisfied and 0 else. Note that in p_1, the randomness is only present in the variables y_n and in p_2 it is in the variables x_n (y is placeholder dummy

used to denote element of $\mathcal{Y}$). Since the RVs are independent and identically distributed, after taking expectation on both sides, we have:

$$\mathbb{E}[p_1(y)] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[\mathbb{I}(y_n = y)] = \frac{1}{N}\sum_{n=1}^{N} p(y_n = y) = p(Y = y) \tag{2}$$

$$\mathbb{E}[p_2(y)] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[p(y|x_n)] = \frac{1}{N}\sum_{n=1}^{N}\sum_{x\in\mathcal{X}} p(Y = y|x_n = x)p(x_n = x)$$

$$= \frac{1}{N}\sum_{n=1}^{N}\sum_{x\in\mathcal{X}} p(Y = y, x_n = x) = \frac{1}{N}\sum_{n=1}^{N} p(Y = y) = p(Y = y) \tag{3}$$

Clearly from the above equations, we can say both the estimates are unbiased. Also both the estimates are sum of IID random variables with finite variance (both $\mathbb{I}(y_n = y)$ and $p(Y = y|x_n)$ are RVs bounded within $[0, 1]$ and hence, cannot have infinite variance). Thus, by Strong law of large numbers, they will converge to the true PMF $p(Y = y)$ almost surely as $N \to \infty$. □

Theorem A.2 (p_2 is the conditional expectation of p_1 conditioned on S_x)**.**

Proof. Starting with the expression for p_1, we have:

$$p_1(y) = \frac{1}{N}\sum_{n=1}^{N}\mathbb{I}(y_n = y)$$

Taking conditional expectation with respect to S_x on both sides, we have

$$\mathbb{E}[p_1(y)|S_x] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[\mathbb{I}(y_n = y) \mid x_1,\ x_2,\ ...,\ x_N] = \frac{1}{N}\sum_{n=1}^{N} p(y_n = y|x_1,\ x_2,\ ...,\ x_N)$$

Since the samples are IID, we have:

$$\frac{1}{N}\sum_{n=1}^{N} p(y_n = y|x_1,\ x_2,\ ...,\ x_N) = \frac{1}{N}\sum_{n=1}^{N} p(y_n = y|x_n) = p_2(y) \tag{4}$$

This concludes the proof. □

Theorem A.3 (KL divergence $D_{\text{KL}}(P||Q)$ is convex in the first argument for a finite sample space)**.**

Proof. Let P_1, P_2, Q be probability distributions over a finite-sized sample set $\mathcal{X}$ where it is also the support of Q, i.e., for any $x \in \mathcal{X}$, $Q(x) > 0$. We have:

$$D_{\text{KL}}(P_1||Q) = \sum_{x\in\mathcal{X}} P_1(x)\log\frac{P_1(x)}{Q(x)}; \quad D_{\text{KL}}(P_2||Q) = \sum_{x\in\mathcal{X}} P_2(x)\log\frac{P_2(x)}{Q(x)}$$

where log is considered over the natural base e. Let $P_3 = \alpha P_1 + (1 - \alpha)P_2$ for some $\alpha \in [0, 1]$. Clearly, P_3 is also a valid distribution over $\mathcal{X}$. We have:

$$D_{\text{KL}}(P_3||Q) = \sum_{x\in\mathcal{X}} P_3(x)\log\frac{P_3(x)}{Q(x)} = \alpha\sum_{x\in\mathcal{X}} P_1(x)\log\frac{P_3(x)}{Q(x)} + (1 - \alpha)\sum_{x\in\mathcal{X}} P_2(x)\log\frac{P_3(x)}{Q(x)}$$

Let $\Delta = D_{\text{KL}}(P_3||Q) - \alpha D_{\text{KL}}(P_1||Q) - (1 - \alpha)D_{\text{KL}}(P_2||Q)$. Then we have:

$$\Delta = \alpha\sum_{x\in\mathcal{X}} P_1(x)\left(\log\frac{P_3(x)}{Q(x)} - \log\frac{P_1(x)}{Q(x)}\right) + (1 - \alpha)\sum_{x\in\mathcal{X}} P_2(x)\left(\log\frac{P_3(x)}{Q(x)} - \log\frac{P_2(x)}{Q(x)}\right)$$

$$= \alpha \sum_{x \in \mathcal{X}} P_1(x) \log \frac{P_3(x)}{P_1(x)} + (1-\alpha) \sum_{x \in \mathcal{X}} P_2(x) \log \frac{P_3(x)}{P_2(x)}$$

Since $\alpha \in [0,1]$, both α and $1-\alpha$ are non-negative. Also both $P_1(x), P_2(x) \geq 0$ since they are probabilities. Hence, using the inequality $\log(x) \leq x-1$, we have:

$$\Delta = \alpha \sum_{x \in \mathcal{X}} P_1(x) \log \frac{P_3(x)}{P_1(x)} + (1-\alpha) \sum_{x \in \mathcal{X}} P_2(x) \log \frac{P_3(x)}{P_2(x)} \leq \alpha \sum_{x \in \mathcal{X}} P_1(x) \left(\frac{P_3(x)}{P_1(x)} - 1 \right) + (1-\alpha) \sum_{x \in \mathcal{X}} P_2(x) \left(\frac{P_3(x)}{P_2(x)} - 1 \right)$$

$$= \alpha \sum_{x \in \mathcal{X}} (P_3(x) - P_1(x)) + (1-\alpha) \sum_{x \in \mathcal{X}} (P_3(x) - P_2(x)) = \alpha(1-1) + (1-\alpha)(1-1) = 0$$

Hence, $\Delta \leq 0 \implies D_{\text{KL}}(P_3||Q) \leq \alpha D_{\text{KL}}(P_1||Q) + (1-\alpha) D_{\text{KL}}(P_2||Q)$ where $P_3 = \alpha P_1 + (1-\alpha) P_2$ and $\alpha \in [0,1]$. This shows the convexity of KL divergence in the first argument for finite-sized sample sets. □

Theorem A.4 (In expectation, the KL divergence between p_2 and p_{true} is less than that between p_1 and p_{true})**.**

Proof. This theorem immediately follows from the above two theorems A.2 and A.3 and then by invoking conditional Jensen's inequality. For notational convenience and minimal clutter, let $L(P)$ denote $D_{\text{KL}}(P||Q)$ for some fixed Q. Since $D_{\text{KL}}(P||Q)$ is convex in P, $L(P)$ is a convex function.

In our problem statement, we have $L(P) = D_{\text{KL}}(P||p_{\text{true}})$, i.e., Q is the true marginal PMF of Y. Hence, using conditional Jensen's inequality, we have:

$$L(p_2) = L\left(\mathbb{E}[p_1|S_x]\right) \leq \mathbb{E}\left[L(p_1)|S_x\right]$$

Taking expectation on both sides and then using the law of total expectation, we get:

$$\mathbb{E}\left[L(p_2)\right] \leq \mathbb{E}\left[\mathbb{E}\left[L(p_1)|S_x\right]\right] = \mathbb{E}\left[L(p_1)\right] \tag{5}$$

and hence, concluding the proof. □

We have shown that in the expected sense of KL divergence, the soft decision based PMF vector estimate is better than the hard decision based empirical estimate. This serves as the main motivation for our idea of using the soft decision vectors in conjunction with Self-Consistency and Self-Certainty methods as explained in the paper.

A.3. Lower Bounding the Jensen Gap

In the previous section, we saw how in expectation, the KL divergence is lower for the soft decisions estimate than the hard decisions based empirical estimate. Jensen's inequality was used to prove that. In this section, we will quantify the Jensen's gap, i.e., the term:

$$\delta = \mathbb{E}[D_{\text{KL}}(p_1||p_{\text{true}})] - \mathbb{E}[D_{\text{KL}}(p_2||p_{\text{true}})]$$

This Jensen's gap can be quantified using the second derivative of the convex function, if it exists (Liao & Berg, 2017):

$$\mathbb{E}[\varphi(X)] - \varphi(\mathbb{E}[X]) \geq \inf_x h(x;\mu) \text{Var}(X)$$

where X is a scalar random variable, $\mu = \mathbb{E}[X]$ is its mean, $\varphi(\cdot)$ is a twice differentiable function and $h(x;\mu)$ is given as:

$$h(x;\mu) = \frac{\varphi(x) - \varphi(\mu)}{(x-\mu)^2} - \frac{\varphi'(\mu)}{x-\mu}$$

The proof is based on the application of second order mean value theorem to the function $\varphi(\cdot)$. Note that if $\varphi(\cdot)$ is convex then $h(x;\mu) \geq 0$ for all $x \neq \mu$ since for convex functions whose first derivative exists, we have:

$$\varphi(x) \geq \varphi(\mu) + \varphi'(\mu)(x-\mu)$$

Inspired from this idea, a similar approach has been followed to quantify the Jensen's gap in our problem. It is easy to see how $D_{\text{KL}}(P||Q)$ is continuous in its first argument over the probability simplex $[0,1]^{|\mathcal{X}|}$ provided $\forall\ x \in \mathcal{X}, Q(x) > 0$. Now let us derive the expressions for its gradient and hessian over the first argument:

$$L(P) = D_{\text{KL}}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$$

$$\Rightarrow \frac{\partial L}{\partial P_a} = 1 + \log \frac{P_a}{Q_a} \tag{6}$$

$$\frac{\partial^2 L}{\partial P_a \partial P_b} = \begin{cases} \frac{1}{P_a} & \text{if } a = b \\ 0 & \text{else} \end{cases} \tag{7}$$

where $P_a = P(X = a)$ denotes the a^{th} element in the PMF vector P. Note that the gradient and hessian exist if $P(x) > 0\ \forall\ x \in \mathcal{X}$, i.e., over the interior $(0,1)^{|\mathcal{X}|}$ of the probability simplex. Let us also define the following term:

$$l_{\text{bound}} = \frac{1}{2N} \left(1 - \mathbb{E}\left[p\left(Y|X\right)\right]\right)$$

With these expressions in hand, the theorem quantifying Jensen's gap in our problem can be derived.

Theorem A.5 (The term l_{bound} lower bounds the Jensen's gap δ in our problem statement)**.**

Proof. In this theorem, we will prove:

$$\delta = \mathbb{E}[D_{\text{KL}}(p_1||p_{\text{true}})] - \mathbb{E}[D_{\text{KL}}(p_2||p_{\text{true}})] \geq \frac{1}{2N} \left(1 - \mathbb{E}\left[p\left(Y|X\right)\right]\right) = l_{\text{bound}}$$

As defined before, $p_2 = \mathbb{E}[p_1|S_x]$ is our soft decisions based estimate. Now let $L(p) = D_{\text{KL}}(p||p_{\text{true}})$ be our convex function over the probability simplex $[0,1]^{|\mathcal{Y}|}$. Let us try to apply second order mean value theorem to this function centered around p_2. We know:

$$p_2 = \frac{1}{N} \sum_{n=1}^{N} p(\cdot|x_n)$$

For all practical purposes, we can assume an LLM assigns non-zero probabilities to all candidate answers irrespective of what the reasoning trace x_n is. The probabilities may be very small but nevertheless non-zero. Hence, the random vector p_2 does not have zero elements, i.e., $p_2 \in (0,1)^{|\mathcal{Y}|}$. Using second order mean value theorem, we have:

$$L(p) = L(p_2) + \nabla L(p_2)^T (p - p_2) + \frac{1}{2}(p - p_2)^T \nabla^2 L(c)(p - p_2) \tag{8}$$

where $p \in [0,1]^{|\mathcal{Y}|}$ is the placeholder variable representing an element of the probability simplex, c is some interior point on the line connecting p and p_2, i.e., $c = \alpha p + (1 - \alpha)p_2$ for some $\alpha \in (0,1)$. Note c will also have only non-zero elements since the contribution from p_2 is always non-zero. Hence, the hessian at c exists and the second order MVT is applicable.

Now let us try to relate the Jensen's gap to this equation. Note that $p_2 = \mathbb{E}[p_1|S_x]$ is a function of S_x. Hence, we have:

$$\mathbb{E}[L(p_1)|S_x] - L(p_2) = \mathbb{E}[(L(p_1) - L(p_2))|S_x] = \int_{(0,1)^{|\mathcal{Y}|}} (L(p) - L(p_2)) P(p_1 = p|S_x) dp$$

where $P(p_1 = p|S_x)$ denotes the probability of the PMF p_1 attaining value p given S_x (since p_1 is a function of the dataset S_y which is a collection of random variables), dp represents the infinitesimal over the probability simplex. Also, the expectation is only over p_1, since p_2 is deterministic when S_x is given.

Using 8, we have:

$$\mathbb{E}[L(p_1)|S_x] - L(p_2) = \int_{(0,1)^{|\mathcal{Y}|}} \left(\nabla L(p_2)^T (p - p_2) + \frac{1}{2}(p - p_2)^T \nabla^2 L(c)(p - p_2) \right) P(p_1 = p|S_x) dp$$

$$= \nabla L(p_2)^T \left(\int_{(0,1)^{|\mathcal{Y}|}} (p - p_2) P(p_1 = p|S_x) dp \right) + \frac{1}{2} \int_{(0,1)^{|\mathcal{Y}|}} (p - p_2)^T \nabla^2 L(c)(p - p_2) P(p_1 = p|S_x) dp \tag{9}$$

Note that the first term is zero since:

$$\nabla L(p_2)^T \left(\int_{(0,1)^{|\mathcal{Y}|}} (p - p_2) P(p_1 = p|S_x) dp \right) = \nabla L(p_2)^T \left(\mathbb{E}[p_1|S_x] - p_2 \right) = 0 \tag{10}$$

Now let us focus on the second term. From 7, we can see the hessian term $\nabla^2 L(c)$ is a diagonal matrix. Hence, its eigenvalues are just its diagonal terms $\frac{1}{c(y)} > 0$, meaning it is a positive definite matrix. Also note $c(y) \leq 1$ for all $y \in \mathcal{Y}$ since c is a PMF vector. Hence, all the eigenvalues $\frac{1}{c(y)} \geq 1$ meaning $\nabla^2 L(c) - I$ is a positive semi-definite matrix. Hence, we have:

$$p^T (\nabla^2 L(c) - I) p \geq 0 \Rightarrow p^T (\nabla^2 L(c)) p \geq p^T p \tag{11}$$

for all vectors p. Using 10, 11, equation 9 becomes:

$$\mathbb{E}[L(p_1)|S_x] - L(p_2) \geq \frac{1}{2} \int_{(0,1)^{|\mathcal{Y}|}} (p - p_2)^T (p - p_2) P(p_1 = p|S_x) dp$$

Note that the RHS is simply:

$$\frac{1}{2} \int_{(0,1)^{|\mathcal{Y}|}} (p - p_2)^T (p - p_2) P(p_1 = p|S_x) dp = \frac{1}{2} \mathbb{E}\left[(p_1 - p_2)^T (p_1 - p_2) | S_x \right]$$

$$= \frac{1}{2} \sum_{y \in \mathcal{Y}} \mathbb{E}\left[(p_1(y) - p_2(y))^2 | S_x \right] = \frac{1}{2} \sum_{y \in \mathcal{Y}} \text{Var}(p_1(y)|S_x)$$

since $p_2(y)$ is simply the conditional expectation of $p_1(y)$ given S_x. Clubbing everything together, we finally get:

$$\mathbb{E}[L(p_1)|S_x] - L(p_2) \geq \frac{1}{2} \sum_{y \in \mathcal{Y}} \text{Var}(p_1(y)|S_x)$$

Taking expectation w.r.t S_x on both sides, we get:

$$\mathbb{E}[L(p_1)] - \mathbb{E}[L(p_2)] \geq \frac{1}{2} \sum_{y \in \mathcal{Y}} \mathbb{E}\left[\text{Var}(p_1(y)|S_x) \right]$$

Using law of total variance, the above equation becomes:

$$\mathbb{E}[L(p_1)] - \mathbb{E}[L(p_2)] \geq \frac{1}{2} \sum_{y \in \mathcal{Y}} \left(\text{Var}(p_1(y)) - \text{Var}(\mathbb{E}[p_1(y)|S_x]) \right) = \frac{1}{2} \sum_{y \in \mathcal{Y}} \left(\text{Var}(p_1(y)) - \text{Var}(p_2(y)) \right) \tag{12}$$

This is the key equation. We are almost there, now we just need to analyze the terms on the RHS. For that, let us look back on the expressions for $p_1(y)$ and $p_2(y)$ once again:

$$p_1(y) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(y_n = y)$$

$$p_2(y) = \frac{1}{N} \sum_{n=1}^{N} p(Y = y|x_n)$$

Note both $p_1(y)$ and $p_2(y)$ are averages of N IID random variables. Hence their variances are given as:

$$\text{Var}(p_1(y)) = \frac{1}{N}\text{Var}(\mathbb{I}(Y=y)) = \frac{1}{N}p(Y=y)(1-p(Y=y))$$

$$\text{Var}(p_2(y)) = \frac{1}{N}\text{Var}(p(Y=y|X)) = \frac{1}{N}\left(\mathbb{E}\left[p(Y=y|X)^2\right] - \mathbb{E}[p(Y=y|X)]^2\right)$$

$$= \frac{1}{N}\left(\mathbb{E}\left[p(Y=y|X)^2\right] - p(Y=y)^2\right)$$

From these expressions, we have:

$$\text{Var}(p_1(y)) - \text{Var}(p_2(y)) = \frac{1}{N}\left(p(Y=y) - p(Y=y)^2 - \mathbb{E}\left[p(Y=y|X)^2\right] + p(Y=y)^2\right)$$

$$= \frac{1}{N}\left(p(Y=y) - \mathbb{E}\left[p(Y=y|X)^2\right]\right)$$

Substituting this in 12, we have:

$$\mathbb{E}[L(p_1)] - \mathbb{E}[L(p_2)] \geq \frac{1}{2N}\sum_{y\in\mathcal{Y}}\left(p(Y=y) - \mathbb{E}\left[p(Y=y|X)^2\right]\right) \tag{13}$$

Note we have:

$$\mathbb{E}\left[p(Y=y|X)^2\right] = \sum_{x\in\mathcal{X}}\left(p(y|x)\right)^2 p(x) = \sum_{x\in\mathcal{X}}\frac{p(y,x)^2}{p(x)}$$

Hence, equation 13 becomes:

$$\delta = \mathbb{E}[L(p_1)] - \mathbb{E}[L(p_2)] \geq \frac{1}{2N}\left(\sum_{y\in\mathcal{Y}}p(Y=y) - \sum_{y\in\mathcal{Y}}\sum_{x\in\mathcal{X}}\frac{p(y,x)^2}{p(x)}\right)$$

$$= \frac{1}{2N}\left(1 - \sum_{y\in\mathcal{Y}}\sum_{x\in\mathcal{X}}p(y|x)p(y,x)\right) = \frac{1}{2N}\left(1 - \mathbb{E}\left[p\left(Y|X\right)\right]\right) = l_{\text{bound}} \tag{14}$$

Hence, proven. □

This is an important result since it quantifies the rate at which the soft decisions based estimate converges faster than the hard decisions based estimate in the expected KL divergence sense. Intuitively also it makes sense since:

$$\mathbb{E}\left[p\left(Y|X\right)\right] = \sum_{x\in\mathcal{X}}\left(\sum_{y\in\mathcal{Y}}\left(p(y|x)\right)^2\right)p(x) = \mathbb{E}[T(X)]$$

where $T(X=x) = \sum_{y\in\mathcal{Y}}\left(p(y|x)\right)^2$. Incidentally, this term can also be written as:

$$\mathbb{E}\left[p\left(Y|X\right)\right] = e^{-R_2(Y|X)}; \quad R_\alpha(Y|X) = \frac{1}{1-\alpha}\log\left(\sum_{x\in\mathcal{X}}p(x)\sum_{y\in\mathcal{Y}}p(y|x)^\alpha\right)$$

where $R_\alpha(Y|X)$ is the conditional Rényi entropy of order α as defined in (Hayashi, 2011).

The $T(X)$ term is simply the squared ℓ_2-norm of the conditional PMF vector $P(\cdot|X)$. The more distributed it is (similar to the uniform distribution over $\mathcal{Y}$), the lesser its $T(X)$ value will be and if it is a point-mass distribution (a one-hot vector assigning all the mass to a single value), then $T(X)=1$. Hence if many reasoning traces are not fully certain about their answers, the $T(X)$ terms for them will be lesser than 1 meaning $1-\mathbb{E}\left[p\left(Y|X\right)\right] = 1-\mathbb{E}[T(X)]$ will be higher. Also, if the reasoning traces are uncertain about their answers, then the entropy term $R_2(Y|X)$ will also be high meaning $1-e^{-R_2(Y|X)}$ will also be high, indicating higher Jensen's gap. And it is exactly this uncertainty of the reasoning traces which we wanted to leverage to obtain a more accurate estimate for the marginal PMF $p(Y)$ in our idea.

A.4. Analyzing the Bias-Variance Components of Error

Inspired by the analysis done in (Zhou et al., 2025), we analyze the bias and variance components of different error metrics in this section and also understand how exactly soft-averaging helps in reducing the error.

A.4.1. Mean Squared Error

In Section A.3, we saw how in expectation, the soft-decisions based estimate has lower KL divergence with respect to the marginal answer PMF, than the one-hot vectors based empirical estimate. However, it is not immediately clear as to why this should provide an improvement in accuracy. To understand that, we need to analyze how the error behaves.

For an input query x, let a denote the ground-truth answer. Let (r, z) denote an output reasoning trace. r and z are similar to the random variables X and Y respectively, as defined in A.1. The ground-truth distribution will then be a one-hot distribution, i.e., $p_{\text{GT}}(Z) = \mathbb{I}(Z = a)$. Let $p_{\text{LLM}}(Z)$ denote the marginal answer distribution explained in A.1 and $p_{est}(Z)$ denote an estimate of this distribution (either Self-Consistency or SASC based estimate). Once again, the conditioning on x is assumed to be implicit and dropped for notational convenience.

Let $\hat{a}$ denote the answer chosen by maximizing this estimate, i.e.:

$$\hat{a} = \text{argmax}_z p_{est}(Z = z)$$

With all the notations defined above, the mean squared error (MSE) for input query x can be written as:

$$\epsilon(x) = \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{est}(z) - \mathbb{I}(z = a))^2\right]$$

where $\mathcal{Z}$ denotes the set of all possible final answers. After adding and subtracting $p_{\text{LLM}}(Z = z)$ and expanding the expression, we get:

$$\epsilon(x) = \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{est}(z) - p_{\text{LLM}}(z))^2\right] + \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{\text{LLM}}(z) - \mathbb{I}(z = a))^2\right]$$

$$+2 \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{est}(z) - p_{\text{LLM}}(z))(p_{\text{LLM}}(z) - \mathbb{I}(z = a))\right]$$

Note that the second term is not random since $p_{\text{LLM}}(Z)$ is fixed for a given input x. Hence, we have:

$$\epsilon(x) = \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{est}(z) - p_{\text{LLM}}(z))^2\right] + \sum_{z \in \mathcal{Z}} (p_{\text{LLM}}(z) - \mathbb{I}(z = a))^2$$

$$+2 \sum_{z \in \mathcal{Z}} (p_{\text{LLM}}(z) - \mathbb{I}(z = a)) \mathbb{E}\left[(p_{est}(z) - p_{\text{LLM}}(z))\right]$$

It is easy to see that the third term is zero since $p_{est}(Z)$ is an unbiased estimate of $p_{\text{LLM}}(Z)$ which was already shown in theorem A.1. Hence, we get:

$$\epsilon(x) = \sum_{z \in \mathcal{Z}} (p_{\text{LLM}}(z) - \mathbb{I}(z = a))^2 + \sum_{z \in \mathcal{Z}} \mathbb{E}\left[(p_{est}(z) - p_{\text{LLM}}(z))^2\right] \tag{15}$$

The first term is precisely the **Bias** component of the error, also known as model error since it captures the model's ability to reason to an input query x. The second term is the **Variance** component of the error, also known as estimation error. This term captures how well the method is able to estimate the marginal distribution $p_{\text{LLM}}(Z)$.

The main contribution of the SASC method is that it reduces this estimation error by using the soft-decision vectors. This is because the soft-decisions based estimate is a conditional expectation of the one-hot vectors based estimate as already shown in theorem A.2. And since conditioning reduces variance, the estimation error of the soft-decisions estimate must be lower.

A.4.2. PROBABILITY OF ERROR

MSE error is a suitable metric to analyze but it imposes a very strong condition. It requires $p_{\text{LLM}}(Z)$ to be nearly one-hot, i.e., $\mathbb{I}(Z = a)$. But all we need is that it must be maximized at $Z = a$ so that with very high probability, the correct answer would be picked. Hence, a better metric to analyze is the probability of error itself. To analyze that, let us first define the maximizer of $p_{\text{LLM}}(Z)$:

$$a_0 = \text{argmax}_z p_{\text{LLM}}(Z = z)$$

Let us also define the following events:

- E_1 : The event that $a \neq \hat{a}$
- E_2 : The event that $a \neq a_0$
- E_3 : The event that $a_0 \neq \hat{a}$

An error will be made when event E_1 happens. But note $E_1 \subseteq E_2 \cup E_3$. This is easy to see since we have:

- E_1^c : The event that $a = \hat{a}$
- E_2^c : The event that $a = a_0$
- E_3^c : The event that $a_0 = \hat{a}$

Clearly $E_2^c \cap E_3^c \subseteq E_1^c$ implying $E_1 \subseteq E_2 \cup E_3$. Hence, using union bound argument, we have:

$$\mathbb{P}(\text{Error}) = \mathbb{P}(E_1) \leq \mathbb{P}(E_2) + \mathbb{P}(E_3) = \mathbb{P}(a \neq a_0) + \mathbb{P}(a_0 \neq \hat{a})$$

Once again note that the first term in the RHS is Bias. In fact, event E_2 is deterministic and hence, $\mathbb{P}(a \neq a_0)$ can only be either 0 or 1. If it is 1, i.e., the model's marginal answer distribution is maximized by a wrong answer, we get a vacuous bound. However under the assumption that $a = a_0$, the error probability is directly bounded by the estimation error probability $\mathbb{P}(a_0 \neq \hat{a})$. SASC minimizes this error by providing a better estimate of $p_{\text{LLM}}(Z)$.

A.5. Analysis for a General Weighing Function $W(\cdot)$

Much of the derivations done in the previous sections focus on the case $W(n) = 1$. However, those arguments can be naturally extended to a general weighing function $W(\cdot)$ with a few assumptions. Let $W(n) = W(r_n, z_n)$ be the score/weight assigned to the n^{th} reasoning trace. We will also assume that this score primarily depends on the reasoning sequence r_n. For example, the score could be assigned by a process reward model which would provide heavy emphasis to the reasoning sequence. Or it could be the Self-Certainty metric which is the average of token-wise KL divergences, implying that the contribution from the reasoning sequence r_n will be much more pronounced than the final answer z_n which would only contribute a few tokens.

Based on this assumption, we can say that the score only depends on r_n, i.e. $W(n) = W(r_n)$. Hence, the score obtained using the WOHV framework for an answer a is:

$$s(a) = \frac{1}{N} \sum_{n=1}^{N} W(r_n) \mathbb{I}(z_n = a)$$

And the modified score vector using soft-decisions will be:

$$\hat{s}(a) = \frac{1}{N} \sum_{n=1}^{N} W(r_n) p(z_n = a \mid r_n)$$

Note both are unbiased estimates of the term:

$$l(a) = \mathbb{E}[s(a)] = \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}[W(r_n) \mathbb{I}(z_n = a)] = \mathbb{E}[W(r) \mathbb{I}(z = a)]$$

Score $l(a)$ is intractable and hence, is estimated using sampled output reasoning traces. Note that $l(a)$ would be considered a "good" score if the weighing function assigns high scores to those reasoning sequences r which produce the correct, ground-truth answer with high probabilities. In our work, we assume that this weighing function is **good**. Under that assumption, SASC provides an improvement by providing a cleaner estimate of $l(a)$.

A.6. Mathematical Formulation of the Majority of the Bests Method and Why it Fits the WOHV framework

For an input query x, let the sampled output reasoning traces be $S = \{y_1, y_2, \ldots, y_N\}$ where $y_i = (r_i, z_i)$ denotes the i^{th} reasoning trace. Let the reward function be $g(.)$. In the Majority of the Bests method, multiple bootstrap datasets are sampled with replacement from this set S. Formally, mB outputs are sampled independently with replacement to form datasets $D_k, k \in \{1, 2, \ldots, B\}$ of size m each:

$$D_k = \{y_{j_{1k}}, y_{j_{2k}}, \ldots, y_{j_{mk}}\}; \quad j_{1k}, j_{2k}, \ldots, j_{mk} \sim \text{Unif}(1, 2, \ldots, N)$$

Then from each dataset, a reasoning trace is chosen using Best-of-N selection, i.e:

$$t_k^{\text{best}} = \text{argmax}_t g\left(y_{j_{tk}}\right); \quad y_k^{\text{best}} = y_{j_{t_k^{\text{best}}k}}$$

Once the best candidates for all bootstrap datasets are chosen, a score is assigned to final answers using Self-Consistency:

$$s_{\text{MoB}}(a) = \frac{1}{B}\sum_{k=1}^{B} \mathbb{I}(z_k^{\text{best}} = a) \text{ where } y_k^{\text{best}} = (r_k^{\text{best}}, z_k^{\text{best}})$$

The final answer is then chosen as the answer maximizing this score. But once again, note that Self-Consistency provides an empirical estimate of the distribution of the best answer candidate by sampling multiple bootstrap datasets. Hence, if the analytical expression of that distribution is available, this process can be avoided. The authors indeed derive an analytical expression for this distribution (Rakhsha et al., 2025).

Let the dataset arranged according to the ascending order of the rewards be $S' = \{y_{(1)}, y_{(2)}, \ldots, y_{(N)}\}$. Note that sampling S' with replacement is the same as sampling S with replacement since S' is just a permutation of S. Let D be a bootstrap dataset obtained by sampling S' with replacement m times. Let L be the best candidate obtained by maximizing the reward within the elements of D. Then we have:

$$\mathbb{P}(L = y_{(i)}) = \mathbb{P}(L \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i)}\}) - \mathbb{P}(L \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i-1)}\})$$

Note L is obtained by maximizing a reward and $y'_{(i)}s$ are already arranged in ascending order of the rewards. Hence, we have the following equivalence:

$$L \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i)}\} \iff \text{All elements of } D \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i)}\}$$

Hence we have:

$$\mathbb{P}(L \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i)}\}) = \mathbb{P}(\text{All elements of } D \in \{y_{(1)}, y_{(2)}, \ldots, y_{(i)}\}) = \left(\frac{i}{N}\right)^m$$

since D is obtained by sampling a N element set independently m times with replacement. Hence, we get:

$$\mathbb{P}(L = y_{(i)}) = \left(\frac{i}{N}\right)^m - \left(\frac{i-1}{N}\right)^m$$

Now the distribution of the final answer Z can be derived as:

$$\mathbb{P}(Z = a) = \sum_{i:z_{(i)}=a} \mathbb{P}(L = y_{(i)})$$

It is easy to see why $s_{\text{MoB}}(a)$ is an unbiased estimate of $\mathbb{P}(Z = a)$ since:

$$\mathbb{E}[s_{\text{MoB}}(a)] = \frac{1}{B}\sum_{k=1}^{B} \mathbb{E}[\mathbb{I}(z_k^{\text{best}} = a)] = \frac{B}{B}\mathbb{P}(z_k^{\text{best}} = a) = \sum_{i:z_{(i)}=a} \mathbb{P}(y_k^{\text{best}} = y_{(i)}) = \sum_{i:z_{(i)}=a} \mathbb{P}(L = y_{(i)})$$

The last step is true because y_k^{best} is just a random variable representing the best candidate trace for a bootstrap dataset which we generally defined as L. Note that $s_{\text{MoB}}(\cdot)$ also falls under the hard-decision based averaging framework:

$$s_{\text{MoB}}(a) = \sum_{i:z_{(i)}=a} \mathbb{P}(L = y_{(i)}) = \sum_{i=1}^{N} \mathbb{P}(L = y_{(i)}) \mathbb{I}(z_{(i)} = a)$$

Hence, the score vector over the unique generated answers corresponding to with and without soft-averaging will respectively be:

$$s_{\text{MoB}} = \sum_{i=1}^{N} W(i) \mathbb{I}_{z_{(i)}}; \quad s_{\text{soft-MoB}} = \sum_{i=1}^{N} W(i) p_{r_{(i)}}$$

where $W(i) = \mathbb{P}(L = y_{(i)})$ is the score assigned to trace $y_{(i)}$ and $p_{r_{(i)}}$ denotes the soft-decision vector corresponding to reasoning sequence $r_{(i)}$. These equations show how the score assigned by MoB method fits the WOHV framework and hence, performance can be improved using soft-averaging.

B. Appendix - Implementation details

Algorithm 1 SASC Implementation Algorithm

Input: Input query x, $\texttt{LLM}\, p_\theta(\cdot)$
Output: Final answer $\hat{a}$

Run inference to independently sample N reasoning traces from the LLM
$S \leftarrow \{(r_1, z_1), (r_2, z_2), \ldots, (r_N, z_N)\} \sim p_\theta(\cdot|x)$

Form the unique answer set A of size $M \leq N$
$A \leftarrow \texttt{Unique}(z_1, z_2, \ldots, z_N)$

Additional scoring pass to compute soft-decision vectors for each reasoning trace
for $n = 1, \ldots, N$ **do**

  ## Compute and store logits L_n of the reasoning sequence r_n
  $L_n \leftarrow \texttt{LLM}(r_n|x)$

  for $k = 1, \ldots, M$ **do**
    ## Obtain logits $L^{n,k}$ for tokens of answer $a_k \in A$ with respect to reasoning r_n
    $L^{n,k} \leftarrow \texttt{LLM}(a_k|L_n)$

    ## For answer a_k with d_k tokens, convert logits to next token probabilities
    for $i = 1, \ldots, d_k$ **do**
      $p_\theta(a_k(i)|a_k(< i), r_n, x) \leftarrow \texttt{Softmax}(a_k(i), L^{n,k}(i))$
    end for

    ## Compute joint probability of answer tokens
    $p(a_k|r_n) \leftarrow \Pi_{i=1}^{d_k} p_\theta(a_k(i)|a_k(< i), r_n, x)$
  end for

  ## Form the soft-decision vector for reasoning trace r_n
  $p_{r_n} \leftarrow [p(a_1|r_n), p(a_2|r_n), \ldots, p(a_M|r_n)]$
end for

Compute soft-averaged score vector
$s = [s_1, s_2, \ldots, s_M] \leftarrow \sum_{n=1}^{N} W(n) p_{r_n}$

Return the final answer corresponding to the highest score in this vector
$k_{\text{best}} \leftarrow \text{argmax}_{k \in \{1,2,\ldots,M\}} s_k$

Return: $\hat{a} \leftarrow a_{k_{\text{best}}}$

B.1. Computational Overheads: Big-O Complexity and Wall-Clock Latencies

To implement soft-averaging, N soft-decision vectors must be computed in an additional scoring pass. Computing these vectors means computing at most N probability terms for all reasoning sequences and hence, the complexity is $O(N^2)$. But in practice, the number of distinct answers M compared to N is relatively small and also the logits corresponding to reasoning r_n can be computed once and reused, as shown in algorithm 1. The logits for answer tokens can be computed in batches, further reducing wall-clock overheads.

Table 4 shows that the overhead imposed by soft-vectors computation is relatively small (around 8-20% of the inference latency) and comparable to the overhead imposed Self-Certainty computation. Moreover, to improve the performance of vanilla Self-Consistency method, more reasoning traces have to be sampled per question, which is a costlier operation.

Table 4. Wall-clock latencies of different tasks using Llama-3.2-3B-Instruct model with $N = 16$ and first 10 questions considered from each dataset

Task	GSM8K	MATH-500	CRUXEval	GPQA-Main
LLM Inference for Self-Consistency	1m 55s	3m 20s	4m 7s	20m 18s
Self-Certainty computation overhead	20s	24s	18s	27s
Soft-Vectors computation overhead	25s	30s	21s	22s

B.2. Computational Advantage of Budgeted Scenarios

In this section, we demonstrate why the reduced reasoning budget scenarios are important from a computational resource saving point-of-view. We do that by showing that when reasoning budgets are soft-enforced, there is a reduction in inference latencies and the average number of output tokens produced.

Table 5. Wall-clock latencies to run inference on the first 10 questions with $N = 16$ using the Qwen-2.5-3B-Instruct model

Soft-enforced reasoning budget	GSM8K	MATH-500	CRUXEval	GPQA-Main
100	25s	33s	25s	30s
500	32s	1m 20s	35s	50s
No Limit	1m 29s	3m 8s	2m 9s	3m 12s

Table 6. Wall-clock latencies to run inference on the first 10 questions with $N = 16$ using the Qwen-2.5-7B-Instruct model

Soft-enforced reasoning budget	GSM8K	MATH-500	CRUXEval	GPQA-Main
100	28s	36s	53s	39s
500	37s	1m 20s	50s	1m 28s
No Limit	52s	1m 33s	1m 6s	2m 2s

Tables 5 and 6 show the reduction in inference latencies under resource-limited scenarios, highlighting the usefulness of these scenarios.

Table 7. Observed average lengths (in tokens) of reasoning traces generated using the Qwen-2.5-3B-Instruct model

Soft-enforced reasoning budget	GSM8K	MATH-500	CRUXEval	GPQA-Main
100	85.79	96.75	71.55	93.16
500	119.97	192.60	119.86	167.15
No limit	237.41	529.99	207.82	437.06

Table 8. Observed average lengths (in tokens) of reasoning traces generated using the Qwen-2.5-7B-Instruct model

Soft-enforced reasoning budget	GSM8K	MATH-500	CRUXEval	GPQA-Main
100	88.24	114.17	74.17	123.79
500	150.37	309.45	138.88	283.84
No limit	190.99	437.19	181.69	471.36

Note that we soft-enforce the reasoning budget via prompting and thus, actual average reasoning trace lengths might be different. Tables 7 and 8 capture those numbers. Moreover, they show that in resource-limited scenarios, average trace lengths are shorter, thereby requiring fewer computational resources. These observations motivated us to consider resource-constrained scenarios for the strong Qwen models and to study how SASC provides an improvement in performance on those models under such scenarios.

C. Appendix - Extra Results

C.1. Results for Llama-3.1-8B-Instruct

Table 9. Accuracies compared across Self-Consistency and Self-Certainty methods for Llama-3.1-8B-Instruct

Method	**GSM8K**		**MATH-500**		**CRUXEval**		**GPQA-Main**	
	$N=8$	$N=64$	$N=8$	$N=64$	$N=8$	$N=64$	$N=8$	$N=64$
Self-Consistency	90.45	91.58	55.40	60.20	47.13	48.88	**33.04**	32.81
SASC (**Ours**)								
• Unnormalized	89.99	91.43	56.60	60.40	47.75	**50.00**	30.58	**33.04**
• Normalized	**90.75**	**91.66**	**57.60**	**61.00**	**48.25**	49.75	30.80	32.81
Self-Certainty								
- $p=0.4$	90.98	91.58	57.20	60.80	47.13	48.88	31.47	33.93
- $p=1.2$	90.98	91.36	55.80	60.60	46.88	49.25	32.37	34.15
- $p=2.0$	90.90	91.36	54.60	**61.40**	45.88	49.25	**32.81**	**35.49**
SASC (**Ours**)								
• Unnormalized								
- $p=0.4$	90.07	91.51	57.20	60.80	**48.13**	49.63	31.03	33.04
- $p=1.2$	90.22	91.43	56.60	60.80	47.00	49.25	32.14	33.48
- $p=2.0$	90.14	91.51	54.80	**61.40**	46.38	49.63	32.59	34.38
• Normalized								
- $p=0.4$	**91.21**	**91.81**	**58.40**	60.80	47.75	49.88	30.13	32.59
- $p=1.2$	90.90	91.51	56.80	61.20	46.88	50.25	32.14	33.71
- $p=2.0$	90.75	91.66	55.40	**61.40**	47.00	**50.38**	32.44	34.60

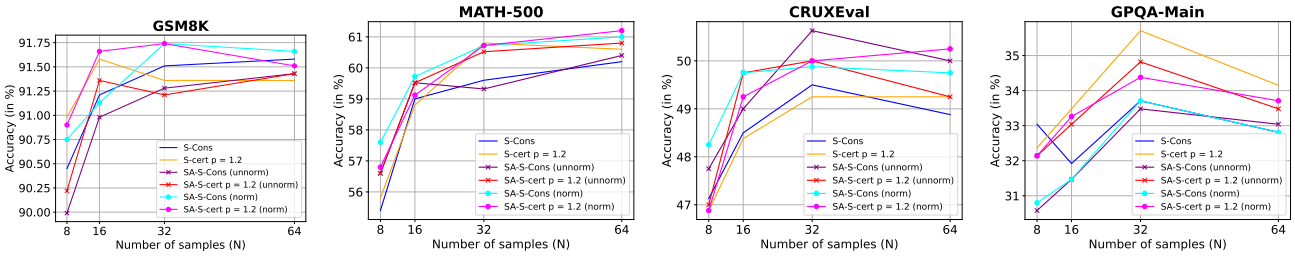

Figure 5. Performance of different methods for $N = 8, 16, 32, 64$ using Llama-3.1-8B-Instruct

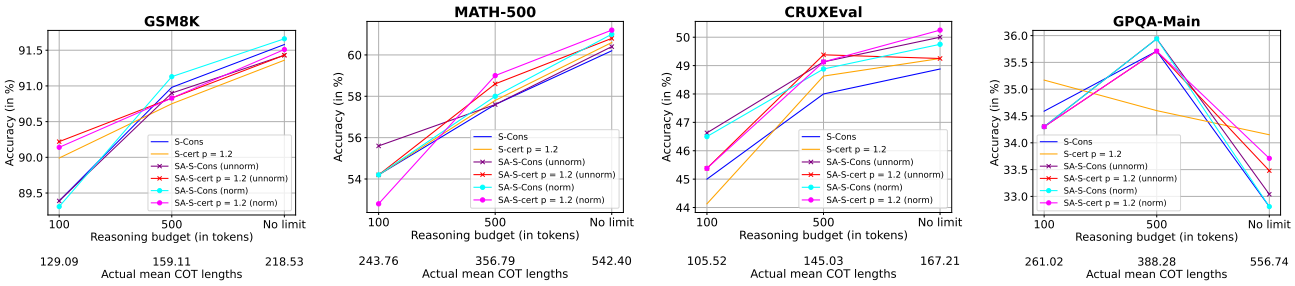

Figure 6. Performance of different methods for reasoning budgets 100, 500 and "No limit" case using Llama-3.1-8B-Instruct

C.2. Entropy Based Analysis of Performance Across Models

From Table 10, we observe that soft-averaging methods provide better performance in both the Llama models (Grattafiori et al., 2024), whereas in Qwen models (Team, 2024; 2025), the performance improvement was marginal, and the better

performer was oscillating between baseline methods and their soft-averaging-based variants.

Table 10. Accuracies compared across Self-Consistency ($p = 0$) and Self-Certainty ($p = 1.2$) methods for different models with $N = 64$ and No limit on reasoning budget

Method	**GSM8K**		**MATH-500**		**CRUXEval**		**GPQA-Main**	
	$p = 0$	$p = 1.2$	$p = 0$	$p = 1.2$	$p = 0$	$p = 1.2$	$p = 0$	$p = 1.2$
Llama-3.2-3B-Instruct								
• Vanilla SC	82.79	83.17	51.00	52.80	37.13	37.75	29.69	29.02
• SASC unnormalized	**83.75**	**84.51**	52.31	53.51	**39.25**	39.38	30.58	30.12
• SASC normalized	82.75	83.36	**52.73**	**54.58**	**39.25**	**39.63**	**30.96**	**31.08**
Llama-3.1-3B-Instruct								
• Vanilla SC	91.58	91.36	60.20	60.60	48.88	49.25	32.81	**34.15**
• SASC unnormalized	91.43	91.43	60.40	60.80	**50.00**	49.25	**33.04**	33.48
• SASC normalized	**91.66**	**91.51**	**61.00**	**61.20**	49.75	**50.25**	32.81	33.71
Qwen3-4B-Instruct								
• Vanilla SC	93.78	**94.01**	**88.93**	**88.52**	**80.51**	**79.62**	54.42	**55.35**
• SASC unnormalized	**93.86**	**94.01**	88.52	88.32	80.38	79.49	**55.12**	**55.35**
• SASC normalized	93.78	**94.01**	88.52	88.32	80.38	79.49	54.42	**55.35**
Qwen2.5-7B-Instruct								
• Vanilla SC	**93.18**	**93.48**	70.94	**71.94**	**61.01**	**60.13**	**35.49**	**35.49**
• SASC unnormalized	92.27	92.49	**71.14**	71.74	60.76	**60.13**	**35.49**	35.27
• SASC normalized	92.04	92.34	70.54	71.74	60.51	60.00	**35.49**	35.27

To investigate the underlying cause of these observations, we considered the Shannon Entropy metric of the normalized conditional PMF vectors as defined in Section 6.1. For a discrete distribution vector $p = [p_1,\ p_2,\ ...,\ p_k]^T$, the Shannon entropy is defined as:

$$H(p) = -\sum_{n=1}^{k} p_n \log(p_n)$$

Shannon entropy is useful in quantifying the randomness present in a distribution. $H(p)$ always satisfies $H(p) \in [0, \log(k)]$ with the minimum achieved by one-hot distributions and the maximum achieved by the uniform distribution. Hence, we considered a scaled version of this quantity as our metric to quantify the nature of the conditional PMF vectors:

$$H_{\text{norm}}(p) = \frac{1}{\log(k)} H(p)$$

$H_{\text{norm}}(p)$ always belongs to the interval $[0, 1]$ and hence the name. In case of $k = 1$, $H_{\text{norm}}(p)$ is set to zero by convention. We computed this metric for different soft-decision vectors obtained from a dataset. Normalization needs to be done since the number of generated unique answers (k) need not be same for all the questions in the dataset and hence, the quantity must be scale-invariant with respect to k.

From Figures 7 and 8, we observe that $H_{\text{norm}}(p)$ is relatively more spread throughout the $[0, 1]$ interval for Llama models than in Qwen models. Even in the Math-500 C-CDF [1] plot, assuming we start looking from $H_{\text{norm}}(p) = 1$ to 0, eventually around $H_{\text{norm}}(p) = 0.55$, the Llama model C-CDFs start to exceed those of Qwen models, exhibiting relatively higher spread. These plots indicate that a significant number of the soft-decision vectors of Llama models are dispersed in nature, as opposed to those of Qwen models which are heavily concentrated around 0, indicating their one-hot behavior. The results present in Table 10 also align with this observation. All these findings agree with our central idea: Using soft-decision

[1] Complementary Cumulative Distribution Function

vectors help in improving performance by estimating the marginal answer PMF better. However if they are already exhibiting one-hot behavior, there will be little to no performance improvement from the baselines.

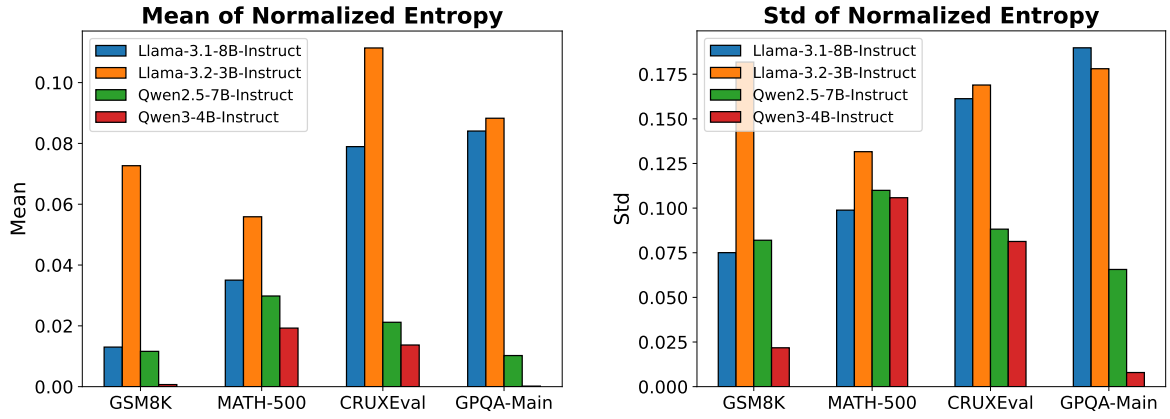

Figure 7. $H_{\text{norm}}(p)$ mean and standard deviation plots. Llama models exhibit relatively higher H_{norm} mean, indicating significant model uncertainty in their answers.

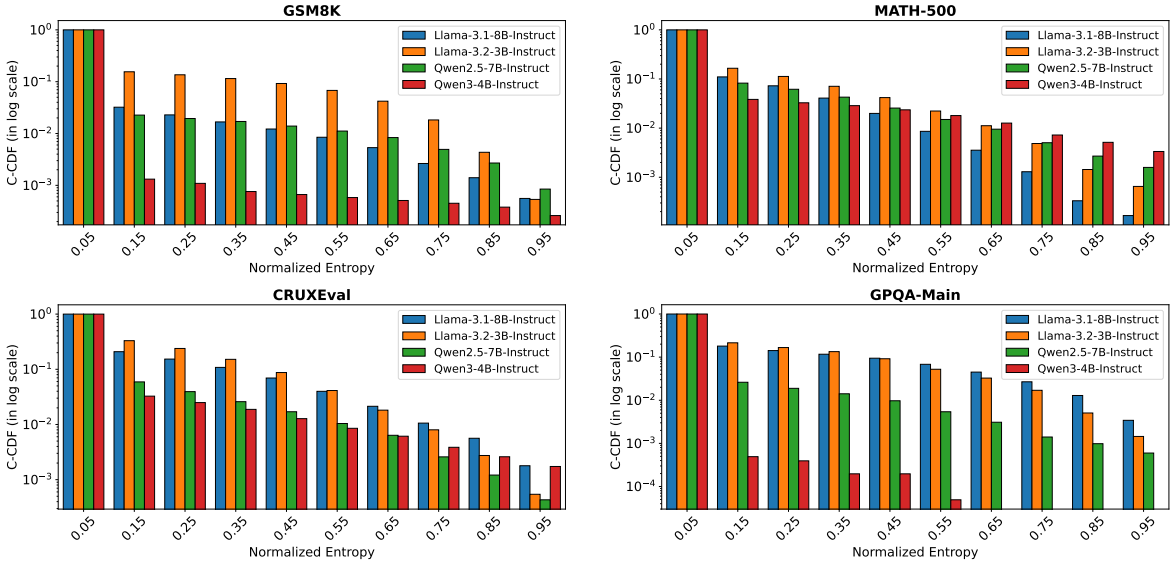

Figure 8. $H_{\text{norm}}(p)$ C-CDF plots for different datasets, models and the "No limit" reasoning budget case. The C-CDFs of Llama models are relatively higher, indicating that their soft-decision vectors are more dispersed in nature. SASC can improve the performance of these models by exploiting the uncertainty in their answers, which is in accordance with the results we see in Table 10.

C.3. Study on Applicability of SASC to Large Reasoning Models (LRM)

LRMs are pre-trained LLMs fine-tuned for reasoning tasks with reinforcement learning based methods such as RLHF (Ouyang et al., 2022), GRPO (Guo et al., 2025). LRMs have shown significant performance improvement over a variety of reasoning tasks in recent times. They achieve this by performing test-time computation to "think" through the problem, generate a sequence of reasoning steps, iteratively refine and self-correct their reasoning process and then generate the final answer. Hence a natural question to ask is: Can soft-averaging improve performance of Self-Consistency/Self-Certainty methods when applied to LRMs? Even though we primarily focused on Instruct-LLMs in this work, we conducted experiments using the Qwen3-4B-Thinking-2507 model (Team, 2025).

Table 11 shows that except in the CRUXEval dataset, soft-averaging either only provides marginal or no improvement over the baseline accuracy. We hypothesized that this result is a consequence of LRMs having high certainty in their answers, i.e., their soft-decision vectors are already near one-hot. To indeed verify our hypothesis, we studied the behavior of the

Table 11. Accuracies compared across Self-Consistency methods for Qwen3-4B-Thinking-2507 LRM

Method	GSM8K	MATH-500	CRUXEval	GPQA-Main
Self-Consistency	96.25	**89.98**	86.33	**79.29**
SASC				
• Unnormalized	**96.33**	**89.98**	88.56	78.70
• Normalized	96.25	**89.98**	**88.71**	**79.29**

soft-decision vectors produced by this LRM. From Figures 9 and 10, it is evident that the LRM model exhibits near one-hot behavior in its soft-decision vectors and thus, will not provide significant performance improvement from the baselines, which is consistent with the results shown earlier.

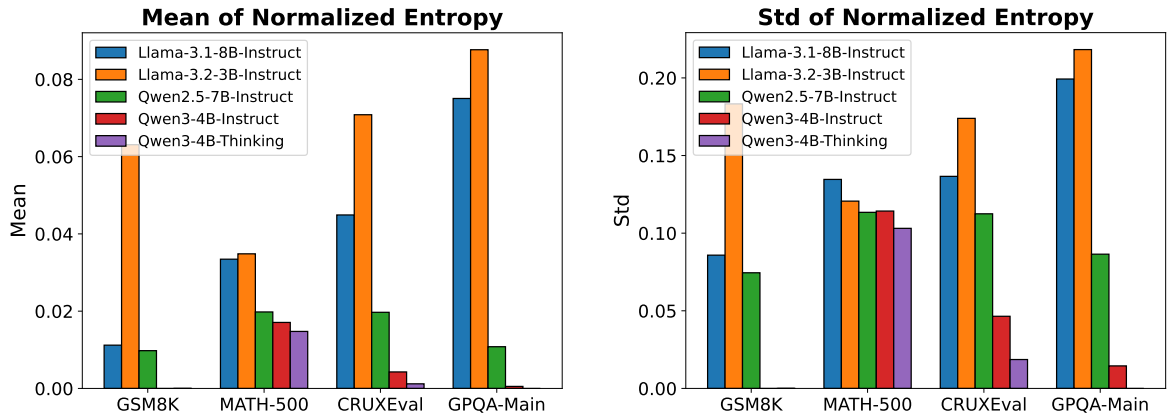

Figure 9. $H_{\text{norm}}(p)$ mean and standard deviation plots including the Qwen3-4B-Thinking-2507 LRM

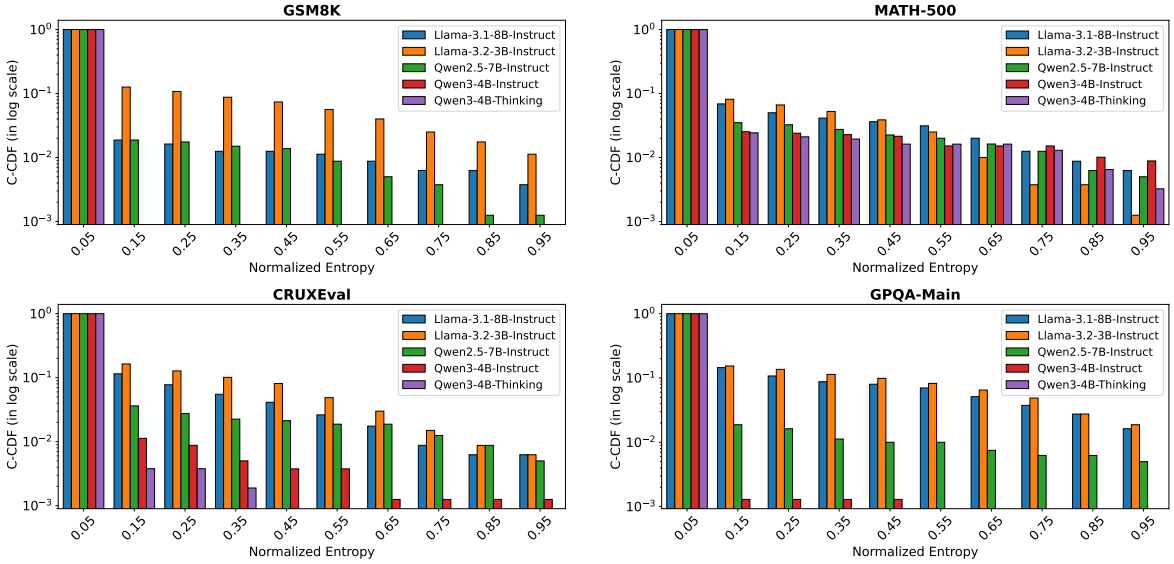

Figure 10. $H_{\text{norm}}(p)$ C-CDF plots for different datasets and models, including the Qwen3-4B-Thinking-2507 LRM. The experiments were conducted with $N = 4$ and on the first 200 examples in each dataset.